

Lab 5: Keypad Scanning and Display

Microprocessors (010153521) | Semester 1/2026

Duration: 3 hours (in-lab) | **Topic:** A 16-function matrix keypad driving an 8-bit two-register calculator (**first**, **second**) shown in hex on a multiplexed 4-digit display – matrix input scanning (Lecture 4) meets AVR arithmetic (Lecture 5)

Objectives

- Understand **matrix keypad scanning** as multiplexed *input* – the mirror image of Lab 4’s multiplexed *output* to a display – and recognize that both need exactly 4 one-at-a-time phases, letting the *same* four MCU pins do double duty.
- Wire a 4x4 matrix keypad, where every key is a dedicated **function** (not a digit), and a 4-digit multiplexed hex display to the Arduino Uno R3.
- Write AVR assembly that decodes a (row, column) key press into one of 16 operations on two 8-bit registers, **first** and **second**, and applies Lecture 5’s arithmetic and shift instructions to update them.
- Extend the provided **hex_to_seg** dispatcher (Lab 4-1’s **DIGIT_TO_SEG** idea, grown to **A-F**) into a full 4-phase multiplexed refresh loop showing **first** and **second** as two 2-digit hex bytes.
- Debounce keypresses and give **first/second** well-defined starting values *before* any arithmetic runs, by dialing them up from 0x00 with the increment/decrement keys.

Quick Reference: Pin Assignment

Port bits	Uno pins	Direction	Function
PB0, PB1	D8, D9	Output	Segments f , a (shared bus, all 4 digits)
PB2-PB5	D10-D13	Output, dual-role	DIGIT1-4 cathodes (direct-drive, no transistor – see Current Budget box) <i>and</i> keypad rows R1-R4 , toggled between input (off) and output-low (on) – see “Digit-Select & Row Scan” below
PC0-PC3	A0-A3	Input, pull-up	Keypad columns C1-C4
PC4, PC5	A4, A5	–	Spare (unused)
PD2, PD3	D2, D3	Output	Segments b , c (shared bus)
PD4	D4	Output	Decimal point (optional, unused by this lab)
PD5-PD7	D5-D7	Output	Segments d , e , g (shared bus)

The segment bus is split across two ports – **a,f** on PORTB, **b,c,d,e,g** on PORTD – because PORTB’s other bits are busy with digit-select/rows and all six of PORTC is reserved for keypad columns. Unlike Lab 4-1’s tidy single-port bus, writing a digit here means one OUT PORTB *and* one OUT PORTD per refresh, not just one.

Quick Reference: Digit-Select & Row Scan via DDR Toggle

PB2-PB5 do two jobs with *one* identical trick, not two opposite ones: toggle the pin’s **direction** (DDR) and leave its PORT bit permanently 0.

- **Input** (DDR bit = 0): the pin floats (high-impedance). A digit’s cathode then has no path to ground, so it’s dark, *and* that row is not being driven, so a key pressed on it can’t pull any column low. Both effects are “off,” for free.
- **Output** (DDR bit = 1, PORT bit = 0): the pin actively sinks to **GND**. The digit’s cathode

is grounded, so whatever pattern is already on the segment bus lights up, *and* that row is driven low, so a pressed key on it pulls its column low. Both effects are “on,” together, from one SBI/CBI DDRB, n .

Per phase i (pin PB(2+ i)): write digit i ’s hex pattern to the segment bus *first*, then SBI DDRB, (2+ i) (digit lights up already correct, no blank/stale flash), poll PINC once for row i ’s keys, hold for t_{dwell} , then CBI DDRB, (2+ i) before moving to phase $i+1$. SBI/CBI need a compile-time constant bit number, so this is written as 4 explicit unrolled blocks, not a loop indexed by a register – see the Starter Code below.

Quick Reference: How Matrix Scanning Works

A 4x4 keypad packs 16 keys into just 8 wires – 4 rows + 4 columns – because each key does nothing but bridge *one* row wire to *one* column wire when pressed; there is no other connection inside the keypad. Driving all 4 rows at once would make every key on a pressed column look identical, so scanning removes the ambiguity: drive exactly *one* row **LOW** (the other three stay floating/inactive), read all 4 columns, and only trust what you read while that one row is the low one. Cycle through all 4 rows, fast enough that a keypress during any row’s turn gets caught.

Quick Reference: Function Key Map

Unlike a calculator’s 0-9 keypad, *every* key on this board is a dedicated operation on one of two 8-bit registers: **first** (columns C1-C2) or **second** (columns C3-C4). **Confirm this against your physical board’s silkscreen/labels** before trusting any decode – this table matches this lab’s reference board.

	C1	C2	C3	C4
R1	S1: inc first	S2: dec first	S3: inc second	S4: dec second
R2	S5: inc inc first	S6: dec dec first	S7: inc inc second	S8: dec dec second
R3	S9: first = first « 1	S10: first = first « second	S11: second = second « 1	S12: second = second « first
R4	S13: first = first + second	S14: first = first - second	S15: second = second - first	S16: second = second + first

“inc inc” / “dec dec” means *call the increment/decrement routine twice*, not a separate +2 instruction – reusing R1’s single-step routine is the point. All eight arithmetic keys (R3-R4) are ordinary 8-bit AVR ops: **first/second** wrap silently on overflow/underflow (mod 256) exactly as ADD/SUB leave them, with the C flag recording what wrapped – see Exercise 5 and the take-home Trace problem.

The four rows split into two structurally different kinds of key. R1-R2 are **single-register** ops – each key touches only **first** *or* only **second**, never both, so they’re the only keys that make sense on a freshly-reset board where both registers are still 0x00. R3-R4 are **combining** ops – every one reads *both* registers to update one of them (a shift by the other register’s amount, or an add/subtract between the two), so they only do something meaningful once both already hold real values. That split is exactly why Exercise 3 (input phase) uses only R1-R2, and Exercise 4 (dispatch table) is where R3-R4 first do anything.

Quick Reference: Hex Segment Codes

Digits 0–9 use the classic **gfedcba** bit order (bit 0 = segment **a** ... bit 6 = segment **g**). This lab's display can also show A–F, since **first/second** are full bytes (0x00–0xFF), not decimal counters:

Digit	0	1	2	3	4	5	6	7	8	9	A	b	C	d	E	F
Hex	3F	06	5B	4F	66	6D	7D	07	7F	6F	77	7C	39	5E	79	71

b and **d** are drawn lowercase (segments **c,d,e,f,g** and **b,c,d,e,g** respectively) so they don't read as 8 and 0 on a 7-segment digit – **A, C, E, F** stay uppercase since they're unambiguous. The provided `hex_to_seg` dispatcher (Starter Code below) implements exactly this table as a CPI/BRNE chain, extending Lab 4-1's DIGIT_TO_SEG idea with six more branches for A–F.

Quick Reference: Current Budget – Why No Transistors Here, and the Catch

Lab 4-1 used an NPN transistor per digit specifically so the AVR pin only ever sources the transistor's small **base** current, not the segments' combined current. This lab's reference board skips that stage: PB2–PB5 sink each digit's cathode *directly*. That works electrically, but it removes Lab 4-1's safety margin – check it before you trust it.

$$I_{\text{segment}} = \frac{V_{CC} - V_{LED}}{R} \approx \frac{5V - 2V}{330\Omega} \approx 9.1 \text{ mA}, \quad I_{\text{digit (worst case)}} = 7 \times I_{\text{segment}} \approx 64 \text{ mA}$$

Digit 8 lights all 7 segments at once, so its digit-select pin momentarily sinks $\approx 64 \text{ mA}$ – *above* the ATmega328P's 40 mA absolute-maximum per-pin rating, even though it's only for one t_{dwell} at a time. Many boards survive this in practice (real parts have margin beyond their datasheet limits), but it is out-of-spec, not a safe design. Before wiring your board, decide: increase the segment resistors (e.g. $330\Omega \rightarrow 1k\Omega$ cuts I_{digit} to $\approx 21 \text{ mA}$, back under the limit, at the cost of a dimmer display), or reintroduce Lab 4-1's transistor stage on PB2–PB5. Either choice is acceptable for this lab – record which you picked and why.

Quick Reference: Multiplexing and Refresh Rate

Only *one* digit's common cathode is ever grounded at a time. Show each digit for a short **dwell time** t_{dwell} , then switch to the next – fast enough that the eye's flicker-fusion threshold ($\approx 50\text{--}60 \text{ Hz}$) blends all four into one steady image:

$$f_{\text{refresh}} = \frac{1}{N_{\text{digits}} \times t_{\text{dwell}}}, \quad N_{\text{digits}} = 4 \Rightarrow t_{\text{dwell}} \leq \frac{1}{4 \times 60\text{Hz}} \approx 4.17 \text{ ms}$$

This bound is *tighter* than Lab 4-1's 2-digit bound (8.3 ms) – twice as many digits means half the time budget per digit for the same refresh rate. Recompute f_{refresh} for your own t_{dwell} below and confirm you're still comfortably clear of 60 Hz.

Quick Reference: Your Individual Target

As in Labs 3, 4-1, and 4-2, derive your personal targets from your student ID:

$$D = \text{sum of every digit in your student ID}$$

Quantity	Formula
Key debounce	$t_{\text{debounce}} = 15 \text{ ms} + (D \bmod 10) \times 1 \text{ ms}$
Display dwell time	$t_{\text{dwell}} = 1.5 \text{ ms} + (D \bmod 6) \times 0.4 \text{ ms}$
first input target	$F_{\text{target}} = D \bmod 32$ (as 0x...hex on the display)
second input target	$S_{\text{target}} = (3 \times D) \bmod 32$

Worked example (a made-up ID, do not reuse): ID 1029384756123 $\rightarrow D = 51$. Then $t_{\text{debounce}} = 15 + (51 \bmod 10) \times 1 = 16 \text{ ms}$; $t_{\text{dwell}} = 1.5 + (51 \bmod 6) \times 0.4 = 1.5 + 3 \times 0.4 = 2.7 \text{ ms}$ (so $f_{\text{refresh}} \approx 92.6 \text{ Hz}$, clear of the 60 Hz bound); $F_{\text{target}} = 51 \bmod 32 = 19 = 0x13$; $S_{\text{target}} = 153 \bmod 32 = 25 = 0x19$.

Bill of Materials

Qty	Part	Notes
1	Arduino Uno R3 (from Lab 3/4)	Supplies power, ground, and all signal pins
1	4x4 matrix keypad module (8-pin), 16 keys each bound to one function	R1-R4, C1-C4
1	4-digit 7-segment display, common cathode (or 2 dual-digit modules wired as one 4-digit bus)	DIGIT1-2 = first , DIGIT3-4 = second
7	Resistor, 330 Ω (or 1 k Ω – see Current Budget box)	Shared segment resistors a-g (one per segment, not per digit)
–	<i>Optional</i> : 4 NPN transistor + 4 base resistors (Lab 4-1 style)	Only if you chose the transistor option in the Current Budget box
1	Breadboard + jumper wires	

Quick Reference: Display Module Pinout (12-pin, common cathode)

This lab's reference build uses a 4-digit module with this physical pinout (pin numbers count around the package, not left-to-right) – **confirm against your own module's datasheet/silkscreen** before wiring, since 12-pin 4-digit modules vary between manufacturers even when common-cathode.

Pin	Label	MCU pin	Pin	Label	MCU pin
1	E	PD6 (D6)	7	B	PD2 (D2)
2	D	PD5 (D5)	8	D3	PB4 (D12)
3	C	PD3 (D3)	9	D2	PB3 (D11)
4	G	PD7 (D7)	10	F	PB0 (D8)
5	D4	PB5 (D13)	11	A	PB1 (D9)
6	(DP, unused)	–	12	D1	PB2 (D10)

D1-D4 are the four digit *commons* (cathodes) – D1 is this lab's DIGIT1 (leftmost, **first's** high nibble) through D4 (rightmost, **second's** low nibble), matching the “first | second” layout on the display. Pin 6's decimal point is present on the package but unused by this lab – leave it unconnected.

Building the Circuit

1. **Rows.** Wire keypad R1-R4 to PB2-PB5 (Uno D10-D13) – the *same* four pins as digit-select (step 4).

2. **Columns.** Wire keypad C1-C4 to PC0-PC3 (Uno A0-A3). These are **inputs** with the AVR's internal pull-up enabled – no external resistor.
3. **Segment bus.** For all 4 digits, tie *every* digit's same-lettered segment pin together (module pins A,B,C,D,E,F,G – Display Module Pinout box), then run that shared node through one 330 Ω (or 1 k Ω) resistor to the matching MCU pin (a(11)→PB1, f(10)→PB0, b(7)→PD2, c(3)→PD3, d(2)→PD5, e(1)→PD6, g(4)→PD7). 7 resistors and 7 MCU pins drive all 4 digits, not 28.
4. **Digit-select.** Module pin D1 (pin 12, leftmost digit) goes *directly* to PB2, D2 (pin 9) to PB3, D3 (pin 8) to PB4, D4 (pin 5, rightmost) to PB5 – the *same* four pins used for keypad rows R1-R4. No transistor, unless you chose that option in the Current Budget box, in which case wire it exactly as Lab 4-1 did (cathode → collector, emitter → GND, base → 1 k Ω → PB(2+i)). Double-check each PB(2+i) node really does connect to *both* the keypad row *and* the digit's cathode – this is the one wiring step that differs from Lab 4-1.

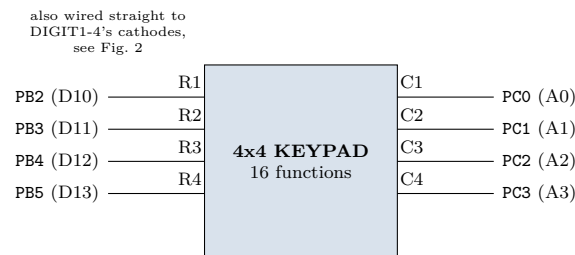


Figure 1: Keypad wiring. Each key merely bridges its row line to its column line when pressed. PB2-PB5 (R1-R4) are dual-role – see Fig. 2 for the digit cathodes sharing these same four pins, no transistor.

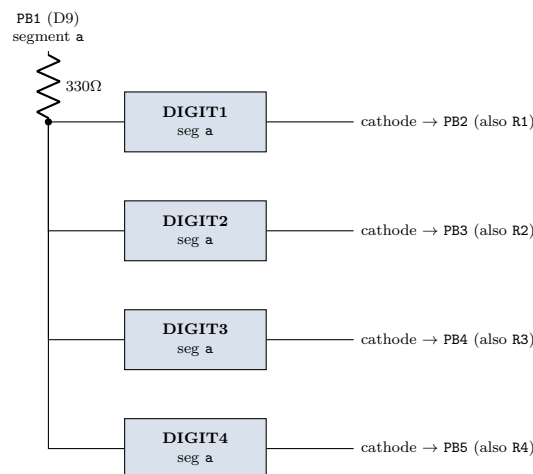


Figure 2: Digit cathodes wired *directly* to the same PB2-PB5 pins used for keypad rows in Fig. 1 – no transistor stage (see Current Budget box). Segment **a** shown representative of the full bus (PB0-PB1, PD2, PD3, PD5-PD7), which repeats identically and is omitted here for readability. DIGIT1-DIGIT2 display **first**; DIGIT3-DIGIT4 display **second**.

Provided Starter Code

The skeleton below is given: the one-time setup, the `hex_to_seg` pattern (shown for digits 0-1 – finishing the rest is Pre-Lab Task 1, next section), `Delay1ms`, and the 4-phase REFRESH loop's shape. Everything marked `TODD` is deliberately unfinished, in three stages: stashing/decoding the column poll into a `key_index` is Pre-Lab Task 2 (also next section, *before* this lab session); debouncing it is Exercise 2; dispatching it to an actual operation on **first/second** is Exercise 4 – the two pieces that are mostly mechanical (getting every segment code right, getting a reliable 16-key decode) are

pushed to homework, so lab time goes to the arithmetic that's this lab's actual point.

```
; ---- one-time setup: segment pins are ALWAYS outputs, never re-cleared ----
sbi  DDRB, 0      ; PB0 = seg f
sbi  DDRB, 1      ; PB1 = seg a
sbi  DDRD, 2      ; PD2 = seg b
sbi  DDRD, 3      ; PD3 = seg c
sbi  DDRD, 5      ; PD5 = seg d
sbi  DDRD, 6      ; PD6 = seg e
sbi  DDRD, 7      ; PD7 = seg g
; PB2-PB5 (digit-select / rows) start as INPUTS (the AVR reset default) --
; do NOT set their DDR bits here; each refresh phase below toggles just its
; own bit, on the way in and back out.
cbi  DDRC, 0      ; PC0-PC3 = keypad columns, input
cbi  DDRC, 1
cbi  DDRC, 2
cbi  DDRC, 3
sbi  PORTC, 0      ; internal pull-ups on
sbi  PORTC, 1
sbi  PORTC, 2
sbi  PORTC, 3

; ---- hex_to_seg: r17 (0x0-0xF) in, segment bus written, r17 clobbered ----
; Same CPI/BRNE-chain idea as Lab 4-1's DIGIT_TO_SEG; 0-9 identical, A-F new.
hex_to_seg:
    cpi    r17, 0x0
    brne   hts_1
    sbi    PORTB, 1      ; a
    sbi    PORTD, 2      ; b
    sbi    PORTD, 3      ; c
    sbi    PORTD, 5      ; d
    sbi    PORTD, 6      ; e
    sbi    PORTB, 0      ; f
    cbi    PORTD, 7      ; g
    ret
hts_1:
    cpi    r17, 0x1
    brne   hts_2
    cbi    PORTB, 1
    sbi    PORTD, 2
    sbi    PORTD, 3
    cbi    PORTD, 5
    cbi    PORTD, 6
    cbi    PORTB, 0
    cbi    PORTD, 7
    ret
hts_2:
    ; ... continue the same pattern for 2-9 ...
    ; then A, b, C, d, E, F using the Hex Segment Codes table above
    ret

; ---- one-time delay tick (~1ms at 16MHz); call it N times for N ms ----
Delay1ms:
    ldi    r18, 21
dl1: ldi    r19, 25
dl2: dec    r19
    brne   dl2
    dec    r18
    brne   dl1
    ret
```

```

; ---- REFRESH: 4 unrolled phases (SBI/CBI need a compile-time bit number, ----
; ---- so this can't be a loop indexed by a register -- see the box above) ----
; r21 = first (0x00-0xFF), r22 = second (0x00-0xFF)
REFRESH:
; ---- Phase 1: first's high nibble, row R1 ----
mov    r17, r21
swap   r17                ; one instruction instead of four LSLs/LSRs
andi   r17, 0x0F
rcall  hex_to_seg
sbi    DDRB, 2            ; DIGIT1 on, ROW1 driven low
in     r16, PINC          ; poll C1-C4 for row 1
; TODO (Pre-Lab Task 2): mask to bits 0-3, compare against 0x0F (the
; "no key in this row" sentinel), and if not equal, fold row+column into
; a single key_index (0-15) and stash it.
rcall  Delay1ms          ; repeat/scale to your t_dwell
cbi    DDRB, 2            ; DIGIT1 off, ROW1 floats

; ---- Phase 2: first's low nibble, row R2 ----
mov    r17, r21
andi   r17, 0x0F
rcall  hex_to_seg
sbi    DDRB, 3
in     r16, PINC          ; row 2
rcall  Delay1ms
cbi    DDRB, 3

; ---- Phase 3: second's high nibble, row R3 ----
mov    r17, r22
swap   r17
andi   r17, 0x0F
rcall  hex_to_seg
sbi    DDRB, 4
in     r16, PINC          ; row 3
rcall  Delay1ms
cbi    DDRB, 4

; ---- Phase 4: second's low nibble, row R4 ----
mov    r17, r22
andi   r17, 0x0F
rcall  hex_to_seg
sbi    DDRB, 5
in     r16, PINC          ; row 4
rcall  Delay1ms
cbi    DDRB, 5

; TODO (Exercise 2): debounce key_index against t_debounce.
; TODO (Exercise 4): dispatch key_index to one of the 16 functions
; (Function Key Map) and update r21/r22 accordingly.

rjmp   REFRESH

```

Pre-Lab Homework (complete before this session)

Two pieces of this lab are almost entirely mechanical – getting every hex digit's segment pattern right, and getting a reliable row/column decode for all 16 keys – and neither one teaches Lecture 5's actual content (arithmetic on *first/second*). Do them now, at home, on *whatever* breadboard/pin layout is convenient – a single-port scheme (e.g. rows and columns both on PORTD, like a tutorial

example) is perfectly fine here, *unlike* the final board. Exercise 1 in lab is specifically about porting a working result onto this lab's real PB2-PB5/PC0-PC3 pin map – that's lab work, not homework. Bring your working `.asm` (or your best attempt, with notes on what's not working) to the lab session.

Task 1 – Finish `hex_to_seg`

Extend the `hex_to_seg` pattern shown in the Starter Code (digits 0 and 1 are done) through every remaining hex digit (2-9, A, b, C, d, E, F), using the Hex Segment Codes table. Test it on a single digit (any breadboard wiring you like) by cycling through all 16 values and confirming each one visually matches the table – the same checkpoint discipline as Lab 4-1's static-digit test, just six more digits.

Task 2 – Full 4-row keypad scan and decode

Write a routine that cycles all 4 rows and, for each row, decides in *one* comparison whether any key in that row is pressed: read the column pins, mask to just the 4 column bits, and compare the result against the “no key” sentinel (all 4 bits still high) – one `ANDI` + one `CPI`, not four separate bit tests. If it's not equal to the sentinel, decode *which* column bit went low and combine it with the current row into a single `key_index` from 0 to 15 (e.g. `key_index = row×4 + column`). Verify by displaying `key_index` in hex on a single digit every time a key is pressed, for all 16 keys – don't worry about functions, debounce, or *first/second* yet, just prove the decode is right for every key.

Why this matters for grading: the in-lab exercises assume Tasks 1-2 already work. If you arrive without them, Exercise 1 (below) is where you'll have to finish them under time pressure, on the harder real pin map – eating into the time budgeted for the arithmetic dispatch table (Exercise 4) that's this lab's actual point.

Quick Experiments (Try It)

Now that Tasks 1-2 work *somewhere*, these two checks confirm they still work on *this* board's specific pins before you rely on them in Exercise 1.

Try It 1: Rows and columns, by hand

Before writing any scanning code, execute `SBI DDRB,2` (Row 1 on, leaving `PORTB,2` at 0) with the other PB3-PB5 bits left as inputs, and columns configured as instructed above. Touch a jumper wire between the R1 and C1 pins on the breadboard (simulating pressing the “inc first” key) and confirm `IN R16,PINC` shows bit 0 (PC0, column 1) go **LOW** only while the wire bridges those two specific pins. Notice `DIGIT1` also lit up the instant you set `DDRB,2` – expected, since it's the same pin doing both jobs.

Try It 2: One row, no full scan

Write a short loop that only ever asserts `DDRB,2` (Row 1) and polls `PINC`. Confirm that keys `inc first`, `dec first`, `inc second`, `dec second` register while keys on other rows (e.g. `inc inc first` or `first = first + second`) do *not* – even though you have not yet written code to scan those other rows. This is your checkpoint that a single phase's scan-and-select works in isolation.

In-Lab Exercises (target: 3 hours)

These assume Pre-Lab Tasks 1-2 already work *somewhere* – lab time goes to porting, debouncing, and dispatching, not to writing the scan/decode from scratch.

In-Lab Exercise 1: Bring-up: port your pre-lab code to this board (30 min)

Wire the full circuit per the instructions above (segments, and PB2-PB5 shared between rows and digit-select, per the Digit-Select & Row Scan box). Port Pre-Lab Tasks 1-2 onto it: `hex_to_seg` should already be complete from Task 1; adapt Task 2's decode to read `PINC` (not whatever port you prototyped with at home) and to fit inside each `REFRESH` phase's brief `SBI DDRB/IN PINC/CBI DDRB` window, rather than a free-running loop.

- Confirm all 4 digits render every hex digit correctly inside the multiplexed `REFRESH` loop (not just a single static digit as in Task 1's home test).
- Confirm all 16 keys still produce the correct `key_index` when displayed raw (e.g. on `DIGIT1`), exactly as in Task 2's home test – now running inside `REFRESH` instead of a standalone loop.
- **Trace/predict first, then verify:** before running it, count the clock cycles your full 4-phase `REFRESH` loop actually takes (`hex_to_seg`'s dispatch chain, the `Delay1ms` calls, the loop overhead) at 16 MHz, and compare the resulting f_{refresh} against your calculated target from the Individual Target box – does the loop's own overhead eat into your dwell budget the way it did in Lab 4-1?
- If Task 1 or 2 isn't fully solid yet, finish it here first – everything below assumes both are working.

In-Lab Exercise 2: Debounce (20 min)

Add software debounce to `key_index` using t_{debounce} from your Individual Target box: after first detecting a non-sentinel column read, wait t_{debounce} and re-scan; only accept the key if the same `key_index` still reads pressed both times. Record in your notebook what you observe *without* debounce (e.g. does a single physical press ever register as more than one `key_index` in a row?) versus with it.

In-Lab Exercise 3: Input phase: set first and second (30 min)

Before any shift/add/subtract key does anything meaningful, both registers need a known starting value. At power-on, `first` and `second` both start at `0x00`. Using *only* the `R1/R2` keys below (the input-phase subset of the full Function Key Map), dial `first` up to your F_{target} and `second` up to your S_{target} from the Individual Target box, confirming each on the display as you go.

	C1	C2	C3	C4
R1	S1: inc first	S2: dec first	S3: inc second	S4: dec second
R2	S5: inc inc first	S6: dec dec first	S7: inc inc second	S8: dec dec second

`R3-R4` (shifts, add, subtract) stay inactive for this exercise – they're Exercise 5's job, once both registers already hold a known value.

- This is the input mechanism for this lab – there is no dedicated 0-9 keypad, so every starting value is built one (or two) steps at a time.
- Record in your notebook how many key presses each target took, and whether using `inc inc/dec dec` instead of `inc/dec` changed that count the way you'd expect.

In-Lab Exercise 4: Function dispatch table (70 min)

With a debounced `key_index` (0-15) now working, build the dispatch: turn `key_index` into a call to the matching operation from the Function Key Map, updating `first` and/or `second`.

This is the lab's actual point – everything before this exercise was plumbing to get here.

- A CPI/BRNE chain (16 branches) or a computed jump (indexed off `key_index`) both work – pick one and justify the choice in your notebook (code size vs. clarity vs. worst-case latency).
- R1-R2's 8 keys should already work from Exercise 3's input phase – this exercise is where R3-R4's 8 keys (shifts, add, subtract) do something for the first time.
- `first = first « second` and `second = second « first` aren't single AVR instructions – write them as a small loop calling LSL the shift-amount's-worth of times, and note in your notebook why AVR has no single opcode for “shift by a register-valued amount” the way some higher-level languages do.

In-Lab Exercise 5: Verify arithmetic wraparound (30 min)

With the dispatch table from Exercise 4 working, exercise the R3-R4 add/subtract keys specifically for their 8-bit overflow/underflow behavior.

- **Trace/predict first, then verify:** set `first = 0xFA` and `second = 0x0A` (re-dial via Exercise 3's method if needed), predict the displayed hex value of `first` after pressing `first = first + second` (S13), using 8-bit wraparound arithmetic – Lecture 5's C flag records exactly this overflow, even though this display only shows the wrapped low byte. Then verify.
- Repeat the same prediction/verify for `second = second - first` (S15) with `second < first`, this time using Lecture 5's C-flag-as-borrow behavior on SUB.

AI Collaboration

Try these prompts with an AI tool:

- “How do I scan a 4x4 matrix keypad in AVR assembly using internal pull-up resistors on the columns?”
- “In AVR assembly, can SBI/CBI take a register-computed bit number, or does it need a compile-time constant – and what's the usual workaround when you wanted a loop indexed by a register?”
- “In AVR assembly, how do ADD and SUB set the carry flag on 8-bit overflow/underflow, and how is that different from the result just silently wrapping?”

Critical check – verify, don't just accept: Before trusting any AI-suggested segment resistor value, check it against your *own* V_{CC} /LED-drop/resistor math (Current Budget box) – a generic answer assumes a specific LED forward voltage that may not match your display's actual datasheet.

Know the limit: AI tools asked to “share a GPIO pin between two peripherals” will often default to a diode-OR or an external mux/decoder chip, without noticing when a much cheaper option exists: two peripherals whose *idle* state is passively safe (floating and driven-low, in this lab's case) can just share the pin directly, as long as you can tolerate their features being mutually exclusive in time. Ask specifically whether your two signals' idle behavior is passively safe before reaching for extra hardware.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: 8-bit wraparound on the display

A student sets `first = 0xF6` and `second = 0x1D`, then presses `first = first + second` (S13). Compute the 9-bit true sum, state what 8-bit value actually lands in `first` (and hence what the display shows), and state whether the AVR's C flag would be set or cleared after the ADD that performs this.

Find the Bug 1: Pull-ups left disabled

A student wired the columns to PC0-PC3 exactly as instructed, correctly set DDRC bits 0-3 to 0 (input), but forgot to set the corresponding PORTC bits to 1 – so the internal pull-ups never turn on. No key is being pressed. Predict what `IN R16, PINC` reads on these four bits, and explain why an unconnected (floating) digital input does not reliably read as either a clean 0 or a clean 1.

Write Code on Paper 1: Hand-write the decode chain

On paper, without a computer, simulator, or AI tool, write the CPI/BRNE comparison chain (or equivalent) that decodes a scan result for *row R2 only* into its four functions (`inc inc first`, `dec dec first`, `inc inc second`, `dec dec second`) – i.e. given that R2 was the row just driven low, and one column bit is known to read low, write the branches that turn “column 1 is low” into calling the increment routine twice on `first`, and so on for columns 2-4.

Draw on Paper 1: Draw the internal matrix

Draw the keypad's internal 4x4 grid from memory: 4 horizontal row lines, 4 vertical column lines, and a switch symbol at each of the 16 crossings, labeled with its function. Then, for the key `first = first + second`, trace and highlight the exact current path from PB5 (R4, driven low) through the pressed switch to PC0 (C1, read low) – and explain, from your drawing, why no *other* column reads low while only that key is held.

Reflection

In one short paragraph, compare this lab's keypad scanning to Lab 4-1's display multiplexing, and note the new twist this lab adds: the *same* four pins (PB2-PB5) now do both jobs, via a single DDR-toggle trick rather than two independently-managed signals. What made that sharing possible (both roles being “float = off, drive-low = on”), and what would have broken it if digit-select had needed an *active-high* scheme (e.g. a common-anode display) instead? Separately, this lab's reference board also skips Lab 4-1's digit-select transistors – connect that choice back to the Current Budget box: was skipping them actually free, or did it just move a cost from the BOM into a constraint on your components (resistor values) and honesty about spec margins? Finally, connect the `first/second` keys back to Lecture 5: which of the 16 functions map directly onto a single AVR instruction (INC, DEC, LSL, ADD, SUB), and which ones (like `first = first « second`) need a small loop instead?